$F2 \parallel w_jC_{\max}$ Structural Properties and Dynamic Programming

Author One[a,*], Author Two[b]

[a]Institution One, Location, Country

[b]Institution Two, Location, Country

Abstract

Minimizing $w_jC_{\max}$ on a two-machine flow shop, denoted $F2 \parallel w_jC_{\max}$, is a classical NP-hard scheduling problem with applications in manufacturing, computing, and logistics. Existing exact methods rely on enumerative techniques that scale poorly with instance size, while approximation algorithms often impose restrictive assumptions on processing times. We first sequence all jobs globally by Johnson's rule, obtaining an initial permutation S. Scanning S from left to right, we partition it into at most K blocks: whenever a weight not seen before appears, it serves as the final job of the current block, and the next job starts a new block. We establish structural properties of optimal schedules: the permutation property, the positional constraint that block $\mathcal{G}_k$ jobs cannot appear beyond the first k blocks, and the block-end condition requiring the final job of each block to carry that block's weight (automatically satisfied by construction). The intra-block Johnson ordering inherited from S is further formalized within the DP framework. Building on these properties and the interval-based dynamic programming framework developed for $F2|r_j|C_{\max}$, we design a polynomial-time dynamic programming algorithm with $O(|Y|_{\max} \cdot n)$ complexity that appends jobs block by block. We further show that geometric rounding of weights and processing times converts this algorithm into a polynomial-time approximation scheme.

Keywords: Flow shop scheduling; $w_jC_{\max}$; Dynamic programming; Structural properties

*Corresponding author. E-mail address: author@email.com

1 Introduction

2 Preliminaries

We consider the problem $F2 \parallel w_j C_{\max}$. Let $\mathcal{J} = \{J_1, J_2, \ldots, J_n\}$ be the set of n jobs to be processed on two machines M1 and M2 in series: each job J_j must be processed first on M1 for a_j time units and then on M2 for b_j time units, with no preemption allowed. M1 can process at most one job at a time, and M2 can process at most one job at a time. Each job J_j has a weight $w_j > 0$. A permutation schedule $S = (\pi(1), \pi(2), \ldots, \pi(n))$ specifies the processing order on both machines. For a permutation S, let $A_i = \sum_{h=1}^{i} a_{\pi(h)}$ be the cumulative M1 completion time after the first i jobs. The makespan $C_{\max}(S)$ is the M2 completion time of the last job, defined recursively by

$$C_i = \max\{\, C_{i-1},\ A_i \,\} + b_i, \quad i = 1, \ldots, n, \tag{1}$$

and$C_0 = 0$, $C_{\max}(S) = C_n$. The objective is to find a permutation S that minimizes $w_j C_{\max}$.

Theorem 2.1. *$F2 \parallel w_j C_{\max}$ is strongly NP-hard.*

Proof. □

In a permutation schedule, the job order on M1 and M2 is identical; for $F2 \parallel w_j C_{\max}$ there always exists an optimal schedule of this form **?**. All n jobs are sequenced by Johnson's rule to obtain a global sequence $S = (\pi(1), \pi(2), \ldots, \pi(n))$ **?**. Scanning S from left to right, we partition it into K blocks by the first occurrence of each new weight: whenever a weight not seen before appears, it serves as the final job of the current block, and the next job starts a new block. Let $W(\mathcal{G}_k)$ denote the weight carried by the final job of block $\mathcal{G}_k$.

We first establish a positional constraint: in any optimal schedule, for each $k = 1, \ldots, K$, the jobs belonging to block $\mathcal{G}_k$ can only appear within the first k blocks of the final schedule. Consequently, once the k-th block of the final schedule has been processed, all jobs from the first k blocks of S are fully scheduled, and the k-th final block may contain only jobs from $\mathcal{G}_k, \mathcal{G}_{k+1}, \ldots, \mathcal{G}_K$. We further establish a block-end condition: in any optimal schedule, the final job of the k-th final block must carry weight $W(\mathcal{G}_k)$. By construction, $W(\mathcal{G}_k)$ is precisely the weight that first appears at the end of $\mathcal{G}_k$ in S, so this condition is automatically satisfied.

Taken together, the permutation property, the positional constraint, and the block-end condition determine the structure of any optimal schedule: all jobs are first sequenced globally by Johnson's rule and partitioned into K blocks by the first occurrence of each new weight; the final schedule respects the block order; and the last job of each block carries the weight $W(\mathcal{G}_k)$.

Example 2.1. *Consider a 6-job instance with Johnson sequence $S = (J_1, \ldots, J_6)$ and weights $(w_1, \ldots, w_6) = (3, 1, 1, 3, 4, 2)$. Scanning S from left to right and cutting whenever a weight not seen before appears yields the following partition:*

$$\underbrace{J_1,\ J_2}_{\mathcal{G}_1}\ \underbrace{J_3,\ J_4,\ J_5}_{\mathcal{G}_2}\ \underbrace{J_6}_{\mathcal{G}_3}$$

The resulting blocks are $\mathcal{G}_1 = \{J_1, J_2\}$, $\mathcal{G}_2 = \{J_3, J_4, J_5\}$, $\mathcal{G}_3 = \{J_6\}$, with block weights $W(\mathcal{G}_1) = 1$, $W(\mathcal{G}_2) = 4$, $W(\mathcal{G}_3) = 2$, where each $W(\mathcal{G}_k)$ is the new weight that triggered the cut.

3 Dynamic Programming Algorithm

The structural properties established in Section 2 enable a dynamic programming algorithm that constructs the schedule by appending jobs block by block. Our approach adapts the interval-based state representation of **?**, originally devised for the $F2|r_j|C_{\max}$, to the $w_jC_{\max}$ objective.

As described in Section 2, the algorithm takes as input the global Johnson sequence $S = (\pi(1), \pi(2), \ldots, \pi(n))$ and its partition into K blocks $\mathcal{G}_1, \ldots, \mathcal{G}_K$, and processes jobs in block order $\mathcal{G}_1, \mathcal{G}_2, \ldots, \mathcal{G}_K$, appending each job to the end of the current partial permutation. The positional constraint is automatically satisfied by this discipline: jobs of $\mathcal{G}_k$ are appended only after blocks $\mathcal{G}_1, \ldots, \mathcal{G}_{k-1}$ have been fully processed. The block-end condition is enforced at block boundaries via the check $W(\mathcal{G}_k) \leq \ell$ described below. Its correctness follows from the fact that $W(\mathcal{G}_k)$ is the weight of the final job of block $\mathcal{G}_k$ and $\ell = W(\mathcal{G}_{k-1})$; the inequality ensures block weights are non-increasing: a block with larger weight appearing later would inflate the weighted makespan $w_jC_{\max}$ beyond the optimum. If violated, no subsequent appending from $\mathcal{G}_{k+1}, \ldots, \mathcal{G}_K$ can alter the already-fixed weight ordering of the prefix, so the state can be safely discarded. If the check passes, ℓ is updated to $W(\mathcal{G}_k)$ for comparison with the next block.

Lemma 3.1. *Within each block $\mathcal{G}_k$, all jobs are sequenced according to the Johnson's rule: for any two jobs $x, y \in \mathcal{G}_k$, job x precedes job y if and only if*

$$\min\{a_x, b_y\} \leq \min\{a_y, b_x\},$$

with ties broken by non-decreasing a_j. Since the initial sequence S already satisfies this ordering globally and each $\mathcal{G}_k$ is a contiguous subsequence of S, the intra-block Johnson order is automatically preserved by the append-in-block-order discipline.

3.1 State Definition and Dynamic Programming

Each state implicitly corresponds to a partial permutation S uniquely determined by the DP construction path; S is not stored. A state is a 4-tuple (A, B, C, ℓ), where A and B

are the cumulative completion times on M1 and M2, respectively, $C = B$ is the makespan of the partial schedule, and ℓ is the reference weight $W(\mathcal{G}_{k-1})$ of the previously completed block. The initial state is $(0,\, 0,\, 0,\, +\infty)$, corresponding to $S = \emptyset$; $\ell = +\infty$ ensures that the first block always passes the weight check.

Let $\mathcal{Y}$ denote the set of states after processing a given prefix of jobs. Given a state $(A,\, B,\, C,\, \ell) \in \mathcal{Y}$ and a job $j \in \mathcal{G}_k$ to append, the transition produces a new state $(A',\, B',\, C',\, \ell')$ defined by

$$A' = A + a_j, \tag{2}$$

$$B' = \max(B,\, A') + b_j, \tag{3}$$

$$C' = B', \tag{4}$$

$$\ell' = \begin{cases} W(\mathcal{G}_k), & \text{if } j \text{ is the last job of } \mathcal{G}_k, \\ \ell, & \text{otherwise.} \end{cases} \tag{5}$$

All four operations take $O(1)$ time. At a block boundary (when the last job of $\mathcal{G}_k$ is appended), the constraint $W(\mathcal{G}_k) \le \ell$ is verified; if violated, the candidate state is discarded.

Lemma 3.2. *At any processing stage, all states in $\mathcal{Y}$ share the same reference weight ℓ. For any two states (A, B, C, ℓ) and (A', B', C', ℓ') in $\mathcal{Y}$ satisfying $C \le C'$, the latter state can be removed from $\mathcal{Y}$.*

Proof. Since all states at the same stage have identical ℓ (they have completed the same set of blocks), the objective value $\ell \cdot C$ is monotone in C. Let S and S' be the partial schedules attaining the two states. Let σ be an arbitrary suffix schedule for the remaining jobs. The contribution of σ to the makespan is monotone non-decreasing in the prefix completion times (A, B); the condition $C \le C'$ guarantees that S leads to a makespan no larger than S' under any suffix, and with identical ℓ, the objective $\ell \cdot C$ is also no larger. Therefore S dominates S'. □

The dominance rule of Lemma 3.2 justifies retaining only states with the minimum C value after processing each job: since all states at a given stage share the same ℓ, minimizing C is equivalent to minimizing the final objective $\ell \cdot C_{\max}$. The complete DP procedure is presented as Algorithm 1.

Algorithm 1 DP_F2_wCmax

1: **Input:** Jobs J with processing times (a_j, b_j) and weights w_j, $j = 1, \ldots, n$.
2: **Output:** The optimal objective value $C^* = \ell \cdot C_{\max}$ and its associated optimal permutation S^*.
3: $S \leftarrow$ JohnsonSort(J)

$seen \leftarrow \{w_{S[1]}\}$; $start \leftarrow 1$; $K \leftarrow 0$
for $i = 2$ to n **do**
  if $w_{S[i]} \notin seen$ **then**
    $K \leftarrow K + 1$; $\mathcal{G}_K \leftarrow S[start \dots i]$
    $seen \leftarrow seen \cup \{w_{S[i]}\}$; $start \leftarrow i + 1$
  end if
end for
if $start \le n$ **then**
  $K \leftarrow K + 1$; $\mathcal{G}_K \leftarrow S[start \dots n]$
end if
$\mathcal{Y} \leftarrow \{(0,\, 0,\, 0,\, +\infty)\}$
for $k = 1$ to K **do**
  $w_k \leftarrow W(\mathcal{G}_k)$
  for $i = 1$ to $|\mathcal{G}_k|$ **do**
    $j \leftarrow \mathcal{G}_k[i]$; $\mathcal{Y}_{\text{new}} \leftarrow \emptyset$
    for each $(A,\, B,\, C,\, \ell) \in \mathcal{Y}$ **do**
      $A' \leftarrow A + a_j$
      $B' \leftarrow \max(B,\, A') + b_j$
      $C' \leftarrow B'$
      if $i = |\mathcal{G}_k|$ **then**
        if $w_k > \ell$ **then continue**
        end if
        $\ell' \leftarrow w_k$
      else
        $\ell' \leftarrow \ell$
      end if
      $\mathcal{Y}_{\text{new}} \leftarrow \mathcal{Y}_{\text{new}} \cup \{(A',\, B',\, C',\, \ell')\}$
    end for
    $C_{\min} \leftarrow \min\{\, C' \mid (\cdot,\, \cdot,\, C',\, \cdot) \in \mathcal{Y}_{\text{new}} \,\}$
    $\mathcal{Y} \leftarrow \{\, s \in \mathcal{Y}_{\text{new}} \mid s.C = C_{\min} \,\}$
  end for
end for
$C^* \leftarrow \min\{\, \ell \cdot C \mid (A,\, B,\, C,\, \ell) \in \mathcal{Y} \,\}$
Recover the optimal permutation S^* by backtracking
return C^*, S^*

3.2 Complexity Analysis

Theorem 3.1. *Algorithm 1 computes an optimal schedule for* $F2 \parallel w_j C_{\max}$ *in time.*

Proof.

□

4 Conclusion